

FOOD STORE

Informe técnico

Primera entrega parcial del TPI

Guillermo Sánchez

Unidades 1, 2 y 3 / 22 de septiembre de 2026

Modelo de datos, integridad y concurrencia, optimización de consultas e implementación de objetos programables en PostgreSQL.

9

objetivos documentados

79

comprobaciones correctas

17.11

PostgreSQL utilizado

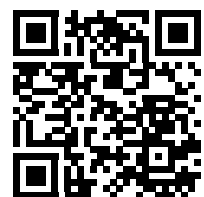
REPOSITORIO Y EVIDENCIAS

github.com/Guille137/Food-Store

Código, scripts, informe y resultados accesibles desde el mismo proyecto. Los enlaces técnicos de este PDF fijan la versión documentada.

Versión técnica: [bf96603](#)

Edición del informe: 1.0 · Documento de entrega



Acceso al repositorio

01 / Resumen técnico

Propósito, implementación por unidad y recorrido de lectura.

La entrega acredita los avances de las unidades 1, 2 y 3 del proyecto Food Store. Integra el trabajo previo y completa el modelo documentado, los procedimientos almacenados y una instalación verificable desde cero. **Es una entrega parcial:** el proyecto continuará en las unidades siguientes.

Unidad 1 • Modelo, integridad y transacciones

Cinco tablas relacionadas mediante PK/FK; modelo ER, transformación relacional y dependencias hasta BCNF. Restricciones CHECK/UNIQUE, triggers de negocio y pruebas de COMMIT, ROLLBACK y aislamiento con conexiones independientes.

Unidad 2 • Consultas y optimización

JOIN, agregaciones, GROUP BY/HAVING, subconsultas y DENSE_RANK. Carga masiva de 600.000 detalles, planes EXPLAIN ANALYZE, índices y reescrituras evaluados por resultados equivalentes y tiempos reales.

Unidad 3 • Índices, vistas y objetos programables

Tres vistas y una materializada, costo de escritura medido, función de total histórico y procedimientos de alta/baja. JSONB como entrada de pedidos y tablas de transición para validar categorías en operaciones de varias filas.

Resultado principal

La instalación independiente completó **79 comprobaciones correctas**, incluidos los ensayos reversibles del DDL. Los registros conservan SQL, parámetros, resultados y errores esperados. No se confunden estas pruebas funcionales con los benchmarks masivos previos.

Guía de lectura

SECCIÓN	SECCIÓN
Cobertura de los nueve objetivos · 03	Modelo ER y cardinalidades · 04
Normalización y DDL · 05	Objetos y reglas de vigencia · 06
Pruebas y concurrencia · 07	Optimización y mediciones · 08
Reproducción, IA y archivos · 09	Mapa de archivos en GitHub

02 / Matriz de cobertura

Los nueve objetivos del enunciado, con acceso directo a su implementación y prueba.

N.º	OBJETIVO	IMPLEMENTACIÓN	EVIDENCIA EN GITHUB
01	Modelo ER	Entidades, atributos, claves, cardinalidades y participación.	Diagrama y diccionario
02	Modelo relacional	FK en relaciones 1:N y tabla DetallePedido para la N:M.	Transformación ER → relacional
03	3FN / BCNF	Dependencias funcionales, claves candidatas y justificación por tabla.	Normalización fundamentada
04	DDL completo	ENUM, TIMESTAMPTZ, IDENTITY, PK/FK, CHECK, UNIQUE e índices.	Esquema · Índices medidos
05	DML y consultas	JOIN, SUM, GROUP BY/HAVING, subconsultas y DENSE_RANK.	Mapa de consultas · Equivalencia
06	Objetos del motor	Vistas, función de total y dos procedimientos PL/pgSQL con CALL.	Vistas · Función · Alta · Baja
07	Reglas de negocio	CHECK, UNIQUE, triggers por fila y triggers con tablas de transición.	Reglas previas · Transición
08	Transacciones	Atomicidad, COMMIT/ROLLBACK, aislamiento y bloqueo entre sesiones.	Pruebas integradoras · Aislamiento TP2
09	Borrado lógico	Bajas de producto/categoría; catálogo, historia e índice parcial coherentes.	Baja y resultados

Trazabilidad de la entrega

Los enlaces apuntan al commit **bf96603**, que conserva la versión técnica evaluada. El repositorio principal puede seguir evolucionando sin alterar los archivos enlazados aquí.

[Registro completo de ejecución](#) · [Informe técnico en Markdown](#)

03 / Modelo de datos

ER con entidad asociativa: cinco tablas, claves y participación explícita.



Clave compuesta: (id_pedido, id_producto) identifica el detalle. Cantidad y precio de venta pertenecen a la relación N:M Pedido–Producto.

Participación: cada pedido tiene un cliente y cada producto una categoría; ambos padres admiten cero o muchos dependientes. Un pedido puede estar vacío en el modelo; el procedimiento de alta exige al menos una línea.

[Modelo completo, tipos y diccionario](#) · [DDL original](#)

04 / Normalización y DDL

La estructura se justifica con reglas del dominio, no con coincidencias de los datos de prueba.

RELACIÓN	CLAVES CANDIDATAS	DEPENDENCIAS FUNCIONALES
Categoría	id_categoria; nombre	Cada clave determina nombre/identificador y activo.
Cliente	id_cliente; correo_electronico	Cada clave determina correo/identificador, nombre, apellido y teléfono.
Producto	id_producto; nombre	Cada clave determina nombre/identificador, precio de lista, stock, activo y categoría.
Pedido	id_pedido	Determina fecha, estado, forma de pago y cliente.
Detalle	(id_pedido, id_producto)	La pareja determina cantidad y precio_unitario. Ninguna parte de la clave basta.

De 1FN a BCNF

1FN. Atributos de valor único y productos de un pedido representados en filas. El JSONB se usa como parámetro de entrada y se transforma en detalles; no sustituye las tablas.

2FN. Las claves de las entidades son simples. En DetallePedido, cantidad y precio de venta dependen de la pareja completa, no de pedido o producto por separado.

3FN / BCNF. Los datos descriptivos permanecen en su entidad y no se duplican por las FK. Bajo las dependencias declaradas, todo determinante no trivial es superclave. Los UNIQUE de nombre/correo son NOT NULL y constituyen claves alternativas.

Precio actual y precio histórico no son el mismo atributo

id_producto no determina el precio_unitario de todas sus ventas. El catálogo guarda precio_lista; el detalle conserva el precio pactado. El total del pedido se calcula y no se almacena de forma redundante.

Restricciones implementadas

IDENTITY para las claves de entidades; PK compuesta en detalle; FK obligatorias con ON DELETE RESTRICT; ENUM para estado y pago; TIMESTAMPTZ para fecha; CHECK para precios/stock no negativos y cantidad positiva; UNIQUE para nombres y correo. Solo el teléfono de cliente admite NULL.

[Demostración por relación](#) · [Definiciones y dominios](#)

05 / Objetos y reglas de negocio

PL/pgSQL, vistas y tratamiento coherente de la vigencia del catálogo.

OBJETO / CÓDIGO	CONTRATO IMPLEMENTADO
Total histórico	Función STABLE. Suma cantidad × precio_unitario; vacío = 0; inexistente = P0002. No elimina bajas del historial.
Registrar pedido	CALL recibe JSONB validado y devuelve id. Copia el precio bajo FOR SHARE; cabecera y líneas son atómicas. El llamador decide COMMIT/ROLLBACK.
Baja de producto	CALL aplica activo=false. Es idempotente, conserva referencias y nombre único; no ejecuta DELETE.
Categorías activas	Dos triggers AFTER INSERT/UPDATE por sentencia usan NEW TABLE. Rechazan categorías inactivas y bloquean su fila durante la validación.

Vistas y seguridad

Catálogo: exige producto y categoría activos. **Pedidos/clientes:** conserva todos los estados y excluye correo/teléfono. **Detalle:** conserva historia y precio de venta aunque el producto deje de estar activo. Las tres se contrastaron con consultas independientes.

El esquema no contiene contraseñas. La prueba de seguridad concede SELECT de la vista a un rol sin acceso a las tablas: la vista se consulta, las tablas se rechazan con 42501 y las columnas de contacto no expuestas, con 42703.

Integridad y borrado lógico

Los triggers heredados exigen pedido PENDIENTE para editar detalles, producto activo y stock suficiente por línea. Solo se permiten PENDIENTE → CONFIRMADO/CANCELADO y CONFIRMADO → ENTREGADO/CANCELADO. No se abandona un estado terminal.

La baja conserva la fila y el historial. La vista controla ambas vigencias, porque un índice parcial de producto solo filtra producto.activo, no una columna de categoría. La validación de categoría en nuevas líneas/ediciones se agrega en esta etapa del TPI.

Alcance explícito

El stock se comprueba por línea; no se reserva ni descuenta automáticamente. CANCELADO es un estado del pedido, no una baja de cliente. Los nuevos objetos son SECURITY INVOKER y no amplían los permisos del llamador.

[Vistas y definiciones](#) · [Resultados de baja lógica](#)

06 / Verificación y concurrencia

Resultados reales de PostgreSQL: instalación nueva, datos de prueba y evidencia conservada.

COMPROBACIÓN	RESULTADO
Alta y total	2 unidades × 10 + 1 × 25 = 45; dos detalles. Pedido vacío: 0.
Atomicidad de CALL	Falla la segunda línea por stock: 23514. No queda cabecera ni primer detalle del CALL fallido.
Validación de entrada	12 casos de JSONB inválido rechazados; pruebas de referencias, pago nulo y salida INOUT inválida.
CHECK / UNIQUE / FK	Rechazos observados: 23514, 23505 y 23503. Transiciones y edición de confirmados controladas.
Bajas e historia	Precio actual cambiado a 999 y producto inactivo: el historial conserva sus valores y total 45.
Operaciones de varias filas	INSERT/UPDATE válidos aceptados; categoría inactiva provoca reversión completa de la sentencia.
Vistas y materializada	EXCEPT ALL bidireccional: (0,0) en las tres vistas. REFRESH CONCURRENTLY refleja la venta entregada.

Secuencia concurrente observada

- 1. A desactiva una categoría y mantiene la transacción sin confirmar.
- 2. B intenta una venta; la observadora detecta el bloqueo con pg_blocking_pids.
- 3. A ejecuta COMMIT. B continúa, recibe 23514 y no deja un pedido parcial.

Otra prueba comprueba visibilidad antes/después de COMMIT y ausencia de cabecera/detalles tras ROLLBACK. Los niveles READ COMMITTED y REPEATABLE READ se acreditan con la evidencia previa del TP2, sin atribuirlos a esta nueva sesión.

79 comprobaciones correctas

Incluyen el ensayo reversible del DDL y la ejecución definitiva; no son 79 reglas distintas. Base final pequeña: 5 categorías, 4.005 productos, 1 cliente, 4 pedidos y 2 detalles. Los benchmarks masivos se mantienen separados.

[Resultados legibles](#) · [Registro SQL/SQLSTATE completo](#) · [Aislamiento TP2](#)

07 / Optimización medida

Tiempos de lectura, costo de escritura y decisiones aceptadas o descartadas.

Medianas de cinco muestras con caché caliente en las sesiones TP4/TP5. La mejora se acepta después de comprobar resultados equivalentes. No se mezclan estas mediciones con la instalación funcional pequeña del TPI.

CONSULTA	ANTES ms	DESPUÉS ms	DECISIÓN
A1 · Facturación por categoría	89,276	64,069	Original con índices; se descarta reescritura más lenta.
A2 · Clientes por gasto	186,607	145,723	Índices + preagregación.
Nombre sin distinguir mayúsculas	9,870	0,024	Seq Scan → Bitmap por índice de expresión parcial.
Confirmados con tarjeta	19,342	2,776	Index Only Scan; evita recorrido y ordenamiento.
Detalle con precio alto	58,102	0,311	Índice por precio histórico; Index Only Scan.
Agregado mensual / materializado	217,223	0,024	39 filas precalculadas; costo de REFRESH separado.

El índice también tiene costo

CARGA DE 800 FILAS	ANTES ms	DESPUÉS ms	VARIACIÓN
INSERT individuales	180,269	188,844	+4,76%
Un INSERT SELECT	47,392	50,960	+7,53%

Siete pares alternados y cargas revertidas. Los INSERT individuales miden tiempo del cliente; el lote, Execution Time del servidor. No se comparan ambos métodos entre sí ni se incluye latencia de COMMIT.

Descarte y mantenimiento

Sobreindexación: se descartó otro índice de detalle por id_pedido; la PK y el índice cubierto previo ya ofrecen ese acceso. **A1:** la reescritura midió 98,570 ms, por encima de 64,069 ms de la original con índices.

Materialización: REFRESH CONCURRENTLY probado en aproximadamente 325–338 ms. Se propone cada 15 minutos y tras el cierre mensual; no se instaló una tarea automática. Entre refrescos o ante una falla, el dato puede quedar atrasado.

[Planes y muestras TP4](#) · [Lectura, escritura y materialización TP5](#)

08 / Reproducción y archivos

Cómo verificar el trabajo y localizar el código, los resultados y las decisiones con IA.

Requisitos: PostgreSQL 16+ iniciado, herramientas pg_dump, Python 3.10+ y usuario local con permiso para crear bases/objetos. Se probó en PostgreSQL 17.11. No hacen falta las bases privadas de los TP anteriores.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe tpi/verificar_tpi.py `
  --pg-bin 'C:/ruta/PostgreSQL/17/bin' `
  --host 127.0.0.1 --port 55432 --user postgres
```

Ejecutar desde la raíz del repositorio en PowerShell y adaptar ruta, puerto y usuario. El script crea una base nueva fechada, respalda antes del DDL, ensaya con ROLLBACK y aplica lo comprobado. Conserva evidencia con **exito: true** o el error si una verificación falla. No instala el servidor ni elimina bases automáticamente.

Uso de IA y decisiones técnicas

Se utilizó **Codex**, conforme a la libertad de herramienta comunicada por el alumno, para especificar, generar SQL/Python, revisar, ejecutar pruebas y documentar. No se atribuye uso a Kiro/OpenCode. Las especificaciones y los commits permiten seguir el proceso.

Aceptado: índices con mejora observada, vistas equivalentes, alta atómica y baja que conserva historia. **Descartado:** índice redundante, reescritura más lenta, total almacenado y borrado físico como baja comercial. El JSONB se transforma en filas normalizadas.

Accesos de entrega

RECURSO	ACCESO
Repositorio principal	Food-Store en GitHub
Versión técnica documentada	bf96603 · Archivos del integrador
Código completo de esa versión	Descargar ZIP desde GitHub
Pruebas y reproducibilidad	Ejecutor · Registro de ejecución
Declaración de IA	DUIA y decisiones

Referencias del motor: [CREATE PROCEDURE](#) · [Transacciones PL/pgSQL](#) · [Triggers y tablas de transición](#)

Este PDF reúne el informe y el acceso al proyecto verificable. Los archivos enlazados forman parte de la entrega técnica; el repositorio conserva el historial de desarrollo y las evidencias de los trabajos anteriores.